სრული ტექნიკური Handoff — NAPR ხარისხის კონტროლის ერთიანი პლატფორმა

> **ვისთვის:** ლუკა ნუკრაძე და სალომე დავითაშვილი (პროექტის ავტორები) — პირადი საცნობარო დოკუმენტი IT-სთან შეხვედრისთვის. **მდგომარეობა:** 2026-10-22. პლატფორმის ვერსია: `0.2.0` (`app/__init__.py`). **საფუძველი:** კოდის, ბაზის, კონფიგურაციისა და ტესტების აუდიტი, რომელიც ამ დოკუმენტის მომზადებისას ჩატარდა (ბაზები მხოლოდ კითხვის რეჟიმში გაიხსნა, კოდი არ შეცვლილა). **დანარჩენი დოკუმენტები:** სარჩევი.

---

0. როგორ წავიკითხოთ ეს დოკუმენტი

0.1 სტატუსის ნიშნები

ყოველ მტკიცებას აწერია, რამდენად დადასტურებულია. ეს ნიშნები ექვსივე დოკუმენტში ერთნაირია.

ნიშანი	მნიშვნელობა	ინგლისური ეკვივალენტი
✅ **დადასტურებული**	კოდში, ბაზაში, ტესტში ან ფაილში შემოწმებულია	VERIFIED
🗓 **დაგეგმილი**	შეთანხმებულია ან კოდში მზადაა ჩასართავად, მაგრამ ჯერ არ მუშაობს	PLANNED
💡 **რეკომენდებული**	ჩვენი ტექნიკური წინადადებაა, ჯერ არავის დაუმტკიცებია	RECOMMENDED
❓ **IT-ის დასადასტურებელი**	დამოკიდებულია IT-ის ინფრასტრუქტურაზე ან გადაწყვეტილებაზე	NEEDS IT CONFIRMATION
⚠️ **შეუსაბამობა**	ორი წყარო ერთმანეთს ეწინააღმდეგება ან დოკუმენტი და კოდი არ ემთხვევა	CONFLICT

0.2 ორი დონის ახსნა

რთულ ადგილებს ორი ახსნა აქვს:

- **ტექნიკურად** — IT-ის ენით, ზუსტი ტერმინებით;
- **მარტივად რომ ავხსნათ** — იგივე, ყოველდღიური ენით.

ეს დოკუმენტი **რა არის და როგორ მუშაობს**-ზეა. ცნებების ღრმა ახსნა (სერვერი, SQL, API, OAuth და ა.შ.) ცალკეა: 02 — პირადი ტექნიკური სახელმძღვანელო.

0.3 რას არ შეიცავს

- პაროლებს, token-ებს, Google ცხრილების იდენტიფიკატორებს, Apps Script-ის მისამართებს;
- პირად ნომრებს, ტელეფონებს, ამზომველების სახელებს;
- რეალურ QC კომენტარებს.

ეს მონაცემი არსებულ პროექტებში ინახება და იქვე უნდა დარჩეს.

---

1. მოკლე შეჯამება

ტექნიკურად: ვაშენებთ როლებზე დაფუძნებულ (RBAC) ვებ-აპლიკაციას — FastAPI (Python), სერვერზე აწყობილი HTML გვერდები (Jinja2 + HTMX) და რელაციური ბაზა (SQLAlchemy 2; პროტოტიპში SQLite, საწარმოოში PostgreSQL). აპლიკაცია ერთ სისტემაში აერთიანებს ოქმების განაწილებას, QC შემოწმებას, ველის პასუხების სამუშაო პროცესს (state machine სრული ისტორიით), სტატისტიკას და Pre-QC-ის შედეგებს. სააგენტოს წყარო სისტემებს მხოლოდ კითხულობს.

მარტივად რომ ავხსნათ: დღეს ხარისხის კონტროლი ხუთ სხვადასხვა ხელსაწყოზეა გაბნეული: Google ცხრილი, მისი სკრიპტები, განაწილების პროგრამა ერთ კომპიუტერზე, სტატისტიკის საიტი და Pre-QC პროგრამა. ვაკეთებთ ერთ საიტს, სადაც ყველა თანამშრომელი შედის საკუთარი ანგარიშით და ხედავს მხოლოდ იმას, რაც მის საქმეს ეხება. ოქმი იქვე ნაწილდება, მოწმდება, ველზე მიდის, ბრუნდება და იხურება, სტატისტიკა კი თავისით ითვლება.

1.1 ერთი წინადადებით — რა მდგომარეობაშია

✅ **ლოკალური პროტოტიპი სრულად მუშაობს რეალურ მონაცემზე:** 44 159 QC შემთხვევა, 605 მომხმარებელი, 7 როლი, სრული სამუშაო პროცესი, განაწილება, სტატისტიკა. 57 ავტომატური ტესტი გადის.

❓ **საწარმოოში გაშვებისთვის IT-ისგან გვჭირდება:**

- სერვერი და PostgreSQL;
- დომენი და HTTPS;
- Google OAuth;
- SMTP;
- წყარო სისტემებზე სერვერიდან წვდომა.

💡 **ჩვენი მხრიდანაც დარჩენილია ტექნიკური სამუშაო** (იხ. §8.3): PostgreSQL-ის დრაივერი და ტესტირება, მიგრაციები, ფონური ამოცანები, რამდენიმე უსაფრთხოების გამკაცრება.

1.2 მთავარი ციფრები (აუდიტი, 2026-10-22)

რა	რაოდენობა	სტატუსი
QC შემთხვევა (ოქმი + საკადასტრო კოდი)	44 159 (სისტემური 41 251, სპორადული 2 908)	✅
ძველი ცხრილის მწკრივი → შემთხვევა	44 538 → 44 159 (379 დუბლიკატი)	✅ (`docs/ARCHITECTURE.md` §9; ბაზაში 44 159 ემთხვევა)
მომხმარებელი ბაზაში	605 (აქტიური როლები: ამზომველი 247, საველე მენეჯერი 14, QC ოპერატორი 14, QC მენეჯერი 2, ადმინი 2, კოორდინატორი 1 და სამსახურის უფროსი 1 — ორივე დემო)	✅
დარეგისტრირებული ელფოსტა	**0** — Google-ით შესვლა ჯერ ვერავის შეეძლება	✅ (მნიშვნელოვანი ხარვეზი)
ავტომატური ტესტი	57 გავიდა (≈ 6 წამი)	✅
ბაზის ზომა (SQLite)	≈ 94 MB + WAL ≈ 4 MB	✅
განაწილების წყარო (REGFIELDALL-ის ლოკალური ასლი)	2 331 017 ჩანაწერი, ≈ 1 GB; ბოლო GDB — 2026-10-19	✅
ხარვეზის კოდი	27 + QC-00 (დროებითი) = 28 ჩანაწერი	✅

---

2. რა პრობლემას ვწყვეტთ

#	დღევანდელი პრობლემა	რატომ არის პრობლემა	როგორ წყვეტს პლატფორმა
1	QC ერთ დიდ Google ცხრილშია: ოპერატორების ჩანართები, სპორადული, სარკისებრი ჩანართები	ცხრილი 30 000-ზე მეტი მწკრივისაა. Apps Script-ს წაკითხვას ~85 წამი სჭირდება და ხანდახან ვარდება (`NAPR_Distributor/README.md`)	✅ ერთი ბაზა ინდექსებით. ჩანაწერი პირდაპირ იძებნება, მთელი ცხრილი არ იკითხება
2	სტატუსი უჯრაში ზედ იწერება	ისტორია იკარგება: ვინ, როდის, რა შეცვალა — უცნობია	✅ ყოველი ცვლილება `qc_events` -ში იწერება და არასოდეს იშლება
3	წესებს ადამიანები იცავენ	მაგალითად, „არ ვეთანხმები“ მენეჯერის დადასტურების გარეშე გადადის, ან ერთ კოდზე ორი პასუხი მოდის	✅ წესები კოდშია: `app/workflow.py` ამოწმებს, ვის რისი უფლება აქვს და რომელ მდგომარეობაში
4	ხილვადობას ცხრილის გაზიარება მართავს	ან ყველა ყველაფერს ხედავს, ან სკრიპტებით „სარკეები“ იქმნება	✅ სერვერი ყოველ მოთხოვნაზე ფილტრავს: `app/scope.py`
5	განაწილების პროგრამა ერთ კომპიუტერზეა (QGIS-ის Python, `Z:\` დისკი)	იმ კომპიუტერის გარეშე განაწილება ვერ ხდება. ჩაწერა ცხრილში სკრიპტით ხდება, ნაწილობრივი ჩავარდნით	✅ განაწილება საიტზეა: ჯერ გადახედვა, მერე ერთი ტრანზაქციით დადასტურება
6	სტატისტიკა Excel → JSON → საიტის ჯაჭვით ითვლება და ხელით ახლდება	ციფრები ყოველთვის დაგვიანებულია და სხვადასხვა ადგილას სხვადასხვაა	✅ სტატისტიკა იმავე ბაზიდან, ერთი ფუნქციით ითვლება ყველა როლისთვის (`app/services/stats.py`)
7	Pre-QC ცალკე პროგრამაა, შედეგები ცალკე ინახება	ველს ერთ ადგილას არ უჩანს Pre-QC-იც და QC-იც	🗓 Pre-QC შედეგები პლატფორმაში ჩანს. ახლა დემოა, რეალური ინტეგრაცია დაგეგმილია
8	სპორადულის სია ყოველდღე მეილით მოდის xlsx-ად, განმცხადებლების პირადი მონაცემებით	ზედმეტი პირადი მონაცემი ვრცელდება	✅ ფაილიდან მხოლოდ საჭირო სვეტები იკითხება, ატვირთული ასლი მაშინვე იშლება. 💡 საწარმოოში — view ან SFTP

მარტივად რომ ავხსნათ: დღეს სამუშაო ცხრილში „იწერება“, მაგრამ სისტემა არ „იცის“, რა ხდება. ახალი პლატფორმა იცის, ვინ ვინ არის, რომელ ეტაპზეა ოქმი, ვის ეკუთვნის შემდეგი ნაბიჯი, და არასწორ ნაბიჯს არ უშვებს.

---

3. არსებული სისტემები — თითოეული ცალ-ცალკე

⚠️ **მთავარი აღმოჩენა:** არსებულ პროექტებს **ერთი არქიტექტურა და ერთი ბაზა არ აქვთ.** თითოეული სხვა ტექნოლოგიითაა აწყობილი, სხვა ადგილას ინახავს მონაცემს და წყაროებსაც სხვადასხვა გზით კითხულობს. ქვემოთ თითოეულია აღწერილი.

3.1 კონტროლის Google ცხრილი და Apps Script

რა არის	დღევანდელი სამუშაო ინსტრუმენტი. თითო QC ოპერატორს თავისი ჩანართი აქვს, არის „სპორადული“ ჩანართი და დამხმარე ჩანართები („შემაჯამებელი“, „არდათანხმებულები“, „_კოდების ცნობარი“ და ა.შ.)
ტექნოლოგია	Google Sheets + Google Apps Script (JavaScript-ის მსგავსი ენა, Google-ის სერვერზე მუშაობს)
სკრიპტები	`qc-dashboard-main/apps_script/` : `QCSyncTrigger.gs` (სინქრონის ტრიგერი), `QCAutoCode.gs` და `rules.generated.gs` (ავტომატური კოდირება), `QCStatRead.gs` (მხოლოდ კითხვა), `QCFillDates.gs` , `QCRestoreNames.gs` , `FixJValidation.gs` ; `NAPR_Distributor/docs/` : `ControlWebApp.gs` (ჩაწერა განაწილებისას), `AppsScript.gs` (Field Group-ის წაკითხვა)
როგორ უკავშირდებიან სხვები	„Web App“ მისამართით და საერთო token-ით. სკრიპტი მფლობელის სახელით სრულდება და JSON-ს აბრუნებს ✅
შეზღუდვები	Apps Script-ს 6-წუთიანი შესრულების ლიმიტი აქვს (ჩვეულებრივ ანგარიშზე); დიდ ცხრილზე წაკითხვა ნელია; ერთდროული ჩაწერა `LockService` -ით იცავს თავს; ისტორია არ ინახება

მარტივად რომ ავხსნათ: ცხრილი არის „რვეული“, სადაც ყველა წერს. სკრიპტები პატარა დამხმარე პროგრამებია, რომლებიც ამ რვეულს კითხულობენ ან ავსებენ. „Web App“ ისეთი ფანჯარაა, საიდანაც სხვა პროგრამას რვეულიდან ინფორმაციის აღება შეუძლია, თუ სწორ კოდს (token-ს) იცის.

ჩვენი Excel-ის მაგალითის სტრუქტურა ✅ (`ექსელის მაგალითი ხხარისხის კონტროლის ერთი შიტი.xlsx` — ერთი ოპერატორის ჩანართი, 2 199 მწკრივი, 23 სვეტი):

სვეტი	სათაური	ახალ ბაზაში
A	ოქმის N	`qc_items.protocol_number`
B	CADCODE	`qc_items.cadcode`
C	QC სპეციალისტი	`qc_items.operator_id`
D	დამატების თარიღი	`qc_items.checked_at` (D — სტატუსის პირველი მინიჭება)
E	შემოწმების სტატუსი	`qc_items.verdict` (მაგალითში: უარყოფითი 838, დადებითი 797, რეკომენდაცია 563, „პროცესშია“ 1)
F, G	ამგეგმავი, ამგეგმავი (2)	`item_people` (`kind = surveyor`)
H	ხარვეზის კოდი	`qc_item_codes`
I	QC კომენტარი	`qc_items.qc_comment`
J	საველე მენეჯერის სტატუსი	`qc_items.field_response`
K	საველე მენეჯერის კომენტარი	`qc_items.field_comment`
L	უკუკავშირის თარიღი	`qc_items.field_responded_at`
M	გასწორდა	`qc_items.corrected_confirmed`
N	ხარვეზია	მოძველებულია, არ გამოიყენება (`legacy_import.py`)
O	უკან დაბრუნებული ხარვეზის კომენტარი	`qc_items.returned_comment`
P, Q	კონტროლი sync, სინქრონის დრო	მხოლოდ გადმოტანისას: „ვისთანაა ოქმი“ წესისთვის
R, S	საველე sync, სინქრონის დრო	იგივე
T	მენეჯერი	`qc_items.field_manager_id` (ახლა Field Group-იდან ითვლება, არა ამ სვეტიდან)
U	ნახაზის შეტანის თარიღი	`qc_items.drawing_date`
V	TYPE	`qc_reviews` (`source = legacy`) — შეზღუდული სტატისტიკა
W	anot	`qc_items.legacy_extra` (ნედლად)

3.2 NAPR_Distributor — ოქმების განაწილების პროგრამა

რა არის	დესკტოპ-პროგრამა (v2.0.0). თარიღის დიაპაზონის ოქმებს ბლოკებად, შემთხვევითობით ანაწილებს QC ოპერატორებზე და შედეგს კონტროლის ცხრილში წერს

ტექნოლოგია	Python, PyQt6 ინტერფეისი, GDAL (GDB-ის წასაკითხად), pandas, openpyxl. გაშვება QGIS 4.x-ის Python-ით ✅
მონაცემის წყარო	ქსელური საქაღალდე `Z:\REGFIELDALL`, სადაც ღამის ექსპორტები `REGFIELDALL_<თარიღი>_01-00_<N>.gdb` სახელით დევს. ფენა `NAPR_VEXP_FLD_GPARCELS_4GDB`. პროგრამა ბოლო GDB-ს თავად პოულობს და გეომეტრიის გარეშე, მხოლოდ საჭირო ველებს ლოკალურ SQLite-ში (`napr_data.sqlite`) ტვირთავს ✅
ლოკალური ბაზა	`napr_data.sqlite` ≈ 1 GB; ცხრილი `parcels` (ველები: `objectid, cadcode, fld_paper_id, app_id, regnumber, is_last, fld_status_name, moxaz_creator, mox_create_date, oqmi_creator, moxazulobis_avtori`) და `import_meta`. ბოლო იმპორტი: `REGFIELDALL_2026-10-19_01-00_93.gdb`, 2 331 017 ჩანაწერი ✅
Field Group	ამზომველი → მენეჯერი Google ცხრილიდან (ჩანართები FIELD_GROUP_272 — დღევანდელი, და სრული — წასულების ჩათვლით), Apps Script-ით ✅
ჩაწერა	კონტროლის ცხრილში `ControlWebApp.gs`-ით, პაროლის დადასტურებით. ჩავარდნისას „რა მოასწრო“ მოწმდება ✅
ალგორითმი	კანდიდატი = კოდი, რომლის **პირველი** ოქმი (უადრესი `MOX_CREATE_DATE`) დიაპაზონშია; „სისტემური სერვისის“ ოქმები იგნორირდება; ღია საველე სტატუსი გამოირიცხება; ბლოკი = კოდი ბოლო სეგმენტის გარეშე (განუყოფელია); მენეჯერების „წყლის შევსებით“ გათანაბრება (`balance.py`) ✅
შეზღუდვები	ერთ კომპიუტერზეა მიბმული (`Z:` დისკი, QGIS); ისტორია არ აქვს; შედეგი Google ცხრილშია

მარტივად რომ ავხსნათ: სააგენტო ყოველ ღამე საველე ოქმების სრულ ამონაწერს ქსელურ საქაღალდეში დებს. განაწილების პროგრამა ამ ამონაწერს კითხულობს, ირჩევს იმ ნაკვეთებს, რომლებიც პირველად აიზომა არჩეულ პერიოდში, ჯგუფებად (ბლოკებად) ყოფს და ოპერატორებს შემთხვევით უნაწილებს. შედეგს Google ცხრილში წერს.

⚠️ **შეუსაბამობა — განაწილების წესი შეიცვალა:** ძველი პროგრამა მენეჯერებს „წყლის შევსებით“ ათანაბრებს (ვისაც ნაკლები აქვს შემოწმებული, მეტი ხვდება). ახალ პლატფორმაში ეს **პროცენტულმა დაფარვამ** ჩაანაცვლა (ნაგულისხმევი ~70% + ზონის/სექტორის/მენეჯერის წესები). გადაწყვეტილება მიღებულია 2026-10-20-ს (`docs/ARCHITECTURE.md` §8.1). შედეგად, ერთსა და იმავე დიაპაზონზე ძველი და ახალი განაწილება ერთნაირ შედეგს **არ** მოგვცემს, და ეს მოსალოდნელია.

3.3 QC Dashboard — სტატისტიკის პანელი

რა არის	ხელმძღვანელობის სტატისტიკის საიტი: KPI-ები, გრაფიკები, მენეჯერებისა და ამზომველების ჭრილი
ტექნოლოგია	Python Flask backend, ECharts frontend; მომხმარებლები username/პაროლით (scrypt hash) ✅
მონაცემის ჯაჭვი	`QC სამუშაო.xlsx` → `build_dashboard.py` → `qc_data.json` → `app.py` → ბრაუზერი. Field Group — `sync_staff.py`-ით ✅
სად მუშაობს	ლოკალურად (`127.0.0.1:5000`). `DEPLOY.md` აღწერს განთავსებას GitHub (private) + Render (უფასო გეგმა) + Neon PostgreSQL-ზე (Frankfurt) ✅ დოკუმენტში. ❓ ამჟამად ნამდვილად მუშაობს თუ არა გარე ღრუბელში — ამ აუდიტით ვერ დადასტურდა
სინქრონი	Apps Script ტრიგერი (ორშ–პარ 9:00, 13:00, 19:00) ✅ დოკუმენტში
შეზღუდვები	მონაცემი ფაილიდან იტვირთება და ხელით ახლდება; Render-ის უფასო გეგმა 15 წუთის უმოქმედობის შემდეგ „იძინებს“ (გაღვიძებას ~50 წამი სჭირდება)

მარტივად რომ ავხსნათ: ეს საიტი თვითონ არაფერს ითვლის „ცოცხლად“. ჯერ Excel-იდან მზა ციფრების ფაილი მზადდება, საიტი კი მხოლოდ ამ ფაილს აჩვენებს. ფაილი რომ განახლდეს, ვიღაცამ უნდა გაუშვას.

⚠️ **IT-სთვის მნიშვნელოვანი:** `DEPLOY.md`-ის მიხედვით, სტატისტიკის პანელის ასლი და მისი მონაცემი (ამზომველების სახელები, QC შედეგები) შეიძლება სააგენტოს გარეთ, უცხოურ ღრუბლოვან სერვისზე იყოს განთავსებული. IT ამაზე შეიძლება იკითხოს. პასუხი: ახალი პლატფორმა **სააგენტოს სერვერზე** იგეგმება (`docs/OPEN_QUESTIONS.md` #5), ხოლო ძველი პანელის ბედი ცალკე უნდა გადაწყდეს ❓.

3.4 Pre-QC Validator — ავტომატური წინასწარი შემოწმება

ეს პროექტი სამი ნაწილისგან შედგება:

ნაწილი	რას აკეთებს	ტექნოლოგია	წყარო	სტატუსი
ლოკალური ვალიდატორი (`app/`)	REDI ოქმის გვერდს აანალიზებს: დოკუმენტის წესები, GIS-შემოწმებები, ფოტოს შემოწმებები, Quality Score (0–100) და სტატუსი `pass / warn / review / fail`	FastAPI, `127.0.0.1:8000`; შეიძლება `.exe`-დ აიწყოს (PyInstaller)	ოქმის შენახული HTML ან ბრაუზერის გაფართოებიდან მიღებული მონაცემი	✅ მუშაობს ლოკალურად
ბრაუზერის გაფართოება (`app/extension/`)	REDI-ის გვერდზე (`https://api.napr.gov.ge/lr/redi*`) ჩაშენდება და მონაცემს ლოკალურ ვალიდატორს (`http://127.0.0.1:8000`) უგზავნის	Chrome Manifest V3	თანამშრომლის **საკუთარი REDI სესია** ბრაუზერში	✅
Hosted (`hosted/`)	შედეგებს ცენტრალურად ინახავს და აჩვენებს; Google-ით შესვლა; ჩაწერა `POST /api/preqc/checks` API გასაღებით	FastAPI + PostgreSQL + Alembic მიგრაციები	ლოკალური ვალიდატორის მზა შედეგები	🛈 მზადაა, **განზრახ არ არის განთავსებული** (`hosted/README.md`)

დამატებითი წყაროები, რომლებსაც ვალიდატორი იყენებს ✅:

- **SLR WFS** — `https://wblr.napr.gov.ge/data/SLR/ows`, ფენა `SLR:GFLD_PARCELS`, პროექცია `EPSG:32638`. ავთენტიფიკაცია: HTTP Basic, მომხმარებელი და პაროლი გარემოს ცვლადებიდან (`WFS_USERNAME`, `WFS_PASSWORD`) (`app/wfs.py`);
- **LR_BUILDINGS WMS** — არჩევითი, მისამართი გარემოს ცვლადშია (`LR_WMS_URL`);
- **REDI-ს ფოტოები** — `https://bs.napr.gov.ge/GetBlob?...` (`app/photo_checks.py`);
- **REDI-ს საკადასტრო ბარათები** — `DYNAMIC.ABSTRACT.PAGE`, `REGCARD` (`app/redi_cards.py`);
- **Anthropic API** — არჩევითი AI-რეკომენდაცია შენიშვნის ტექსტზე. ნაგულისხმევად გამორთულია (`SEMANTIC_AI=off`).

მარტივად რომ ავხსნათ: Pre-QC-ს „თვალები" თანამშრომლის ბრაუზერშია. როცა ამზომველი ან მენეჯერი REDI-ში ოქმს ხსნის, გაფართოება ამ გვერდს „კითხულობს" და იმავე კომპიუტერზე გაშვებულ ვალიდატორს აწვდის. ვალიდატორი რუკის სერვისსაც (WFS) ეკითხება, ნაკვეთი სწორად არის თუ არა დახატული. ყველაფერი მუშაობს იმიტომ, რომ ადამიანი REDI-ში უკვე შესულია და შიდა ქსელშია.

⚠️ **IT-სთვის კრიტიკული:** Pre-QC დღეს **თანამშრომლის სესიითა და კომპიუტერით** მუშაობს. სერვერს ასეთი სესია არ აქვს. ამიტომ, თუ Pre-QC სერვერზე ავტომატურად უნდა ეშვებოდეს, საჭიროა:

- ოქმის მონაცემის API;
- ან ოქმის HTML/TXT ექსპორტი;
- WFS-ზე სერვისის ანგარიში (ადამიანის პირადი ანგარიშის ნაცვლად).

ეს ღია კითხვაა ❓ (`docs/OPEN_QUESTIONS.md` #8). ზოგადი პრინციპი იმის შესახებ, რატომ არ მუშაობს სერვერზე ის, რაც დეველოპერის კომპიუტერზე მუშაობს, ახსნილია სახელმძღვანელოს §C.9-ში.

3.5 შედარება ერთ ცხრილში

	კონტროლის ცხრილი	Distributor	Dashboard	Pre-QC (ლოკალური)	Pre-QC Hosted	ახალი პლატფორმა
ტიპი	Google Sheets	დესკტოპი (PyQt6)	ვებ (Flask)	ვებ, ლოკალური (FastAPI)	ვებ (FastAPI)	ვებ (FastAPI)
ბაზა	ცხრილი თავად	SQLite (≈1 GB ასლი)	JSON ფაილი (+ არჩევით PostgreSQL)	JSONL ფაილი	PostgreSQL	SQLite → 🅿 PostgreSQL
შესვლა	Google-ის გაზიარება	არა (ჩაწერაზე პაროლი)	username/პაროლი	არა (localhost)	Google OIDC	🅿 Google OIDC (ახლა დემო)
წყარო	ადამიანები წერენ	`Z:\REGFIELDALL` GDB + Apps Script	Excel + Apps Script	REDI გვერდი + WFS	ლოკალური ვალიდატორი	Distributor-ის SQLite (`mode=ro`) + Apps Script (GET) + xlsx
წერს წყაროში?	—	**დიახ** (კონტროლის ცხრილში)	არა	არა	არა	**არა** ✅
ისტორია	არა	არა	არა	შედეგების ფაილი	append-only	✅ append-only (`qc_events`)
სად მუშაობს	Google	ერთი PC	PC (+ შესაძლოა Render)	PC	არსად (ჯერ)	PC (localhost / LAN დემო)

---

4. ახალი პლატფორმა — დღევანდელი იმპლემენტაცია

ბილიკი: `C:\Users\LNUKRADZE\Desktop\apisaiti\qc-platforma`. ყველა მტკიცება ამ თავში ✅ **დადასტურებულია კოდით**, თუ სხვა რამ არ წერია.

4.1 ტექნოლოგიები და ვერსიები

ფენა	რა	დაყენებული ვერსია (`.venv`)	რატომ
ენა	Python	3.14.5 (მოთხოვნა: ≥ 3.12)	იგივე ენაა, რაც Distributor-სა და Pre-QC-ში. ერთი გუნდი ერთ ენაზე
ვებ-ჩარჩო	FastAPI / Starlette	0.148.0 / 1.9.0	სწრაფია, ავტომატურ API დოკუმენტაციას (`/docs`) ქმნის, Pre-QC-შიც იგივეა
აპლიკაციის სერვერი	uvicorn	0.54.0	FastAPI-ის სტანდარტული ASGI სერვერი
ORM (ბაზასთან ურთიერთობა)	SQLAlchemy	2.1.5	ერთი კოდი SQLite-სა და PostgreSQL-ზე

ფენა	რა	დაყენებული ვერსია (`.venv`)	რატომ
შაბლონები	Jinja2	3.1.6	HTML სერვერზე იწყობა — build-ის ეტაპი არ სჭირდება
ინტერაქტიულობა	HTMX	2.0.4 (`app/static/vendor/htmx.min.js`)	გვერდის ნაწილები განახლდება JavaScript-ის ჩარჩოს გარეშე
გრაფიკები / რუკა	ECharts / Leaflet	5.5.1 / ლოკალური ასლი	ყველაფერი `app/static/vendor/` -შია — გარე CDN არ გამოიყენება
Excel	openpyxl	3.1.5	ექსპორტი, სპორადულის ფაილი, ძველი ცხრილი
HTTP კლიენტი	requests + truststore	—	Google OAuth; კორპორატიული სერტიფიკატი Windows-ის საცავიდან
სესიის ხელმოწერა	itsdangerous	—	cookie-ს გაყალბებისგან დაცვა
ტესტები	pytest + httpx	—	57 ტესტი

მარტივად რომ ავხსნათ: Python არის ენა, რომლითაც საიტის „ტვინი“ წერია. FastAPI ამ ტვინს ვებ-მოთხოვნების მიღებას ასწავლის. SQLAlchemy ბაზასთან სალაპარაკო „თარჯიმანია“. Jinja2 HTML გვერდებს აწყობს. HTMX გვერდს ცოცხლად ანახლებს, ხელახალი ჩატვირთვის გარეშე. ყველა ბიბლიოთეკა პროექტშივეა, ამიტომ საიტს გარე ინტერნეტი არ სჭირდება ფაილების ჩამოსატვირთად.

4.2 კოდის რუკა

```
app/
  main.py          აპლიკაციის აწყობა: მარშრუტები, უსაფრთხოების თავსართები, შეცდომის გვერდები
  config.py        ყველა პარამეტრი გარემოს ცვლადებიდან (QCP_*)
  db.py            ბაზასთან კავშირი (engine, სესია); SQLite-ზე WAL რეჟიმი
  models.py        27 ცხრილის აღწერა (SQLAlchemy)
  domain.py        ლექსიკონი: როლები, ვერდიქტები, მდგომარეობები, ფერები, წარწერები
  workflow.py      სამუშაო პროცესი (state machine): ვის რა შეუძლია, რა ხდება შემდეგ
  scope.py         ხილვადობა: ვინ რომელ ჩანაწერს ხედავს (ყოველ მოთხოვნაზე)
  security.py      სესია, როლის შეცვლა, CSRF შემოწმება, აუდიტი
  templating.py    Jinja2-ის გარემო
  routes/          გვერდები და API: auth, google_auth, pages, qc, field, items,
                   distribution, stats, preqc, admin, api
  services/        directory (ვინ ვისი გუნდიდანაა), distribution/ (algo, source, service),
                   sporadic, legacy_import, stats, notify (მეილი), codes (ავტომატური კოდირება),
                   taxonomy, people, seed, preqc_demo
  templates/       HTML შაბლონები
  static/          CSS, JS, ვენდორის ბიბლიოთეკები, ზონების GeoJSON, სახელმძღვანელოს PDF
  seed/            ხარვეზის კოდების კატალოგი, მივლინებები
scripts/           fetch_sheets (Google ცხრილები → data/raw), build_db (data/raw → ბაზა),
                   run_local, serve_lan, setup_data, smoke_* (შემოწმებები)
tests/             57 ტესტი დროებით ბაზაზე
data/              🔐 ლოკალური მონაცემი (git-ში არ შედის): platform.db, raw/, uploads/, .session_secret
docs/              ARCHITECTURE, IT_REQUEST, DEMO, OPEN_QUESTIONS, handoff/ (ეს დოკუმენტები)
```

4.3 როგორ მუშაობს ერთი მოთხოვნა

ტექნიკურად:

1. ბრაუზერი აგზავნის HTTP მოთხოვნას, მაგალითად `POST /items/4512/action` , cookie-თი `qcp_session` .
2. uvicorn მოთხოვნას FastAPI-ს გადასცემს. `security.current()` cookie-დან ხელმოწერილ `sid` -ს კითხულობს (`itsdangerous`), `sessions` ცხრილში პოულობს სესიას, ამოწმებს ვადას (12 სთ), გაუქმებას და მომხმარებლის აქტიურობას.
3. `same_origin()` ამოწმებს `Origin` / `Referer` თავსართს (CSRF).
4. `items._load()` ჩანაწერს ტვირთავს და `scope.can_view()` -ით ამოწმებს, აქვს თუ არა ამ როლს მისი ნახვის უფლება. თუ არა — **404**.
5. `workflow.apply()` ამოწმებს:
   - არის თუ არა მოქმედება `available_actions()` -ში ამ როლისთვის და ამ მდგომარეობისთვის;
   - ემთხვევა თუ არა ფორმის `version` ჩანაწერის `version` -ს;
   - სავალდებულო ველებს. შემდეგ ცვლის მდგომარეობას და წერს `QcEvent` -ს.
6. თუ მონიშნულია ✉, `notify.queue()` ქმნის მეილს.
7. `db.commit()` — ყველაფერი ერთ ტრანზაქციაში ინახება. შეცდომისას `rollback` .
8. პასუხი: გადამისამართება (303) შეტყობინებით.

მარტივად რომ ავხსნათ: ღილაკზე დაჭერისას სერვერი რიგრიგობით ამოწმებს:

- ვინ ხარ;
- ამ საიტიდან მოხვედი თუ არა;
- შეგიძლია თუ არა ამ ოქმის ნახვა;
- შეგიძლია თუ არა ეს მოქმედება ამ ეტაპზე;
- ხომ არ შეცვალა ოქმი ვინმემ ამასობაში.

ყველა შემოწმება რომ გაივლის, ცვლილება და მისი ისტორიის ჩანაწერი ერთად ინახება. ან ორივე ინახება, ან არცერთი.

4.4 მონაცემთა მოდელი

27 ცხრილი (`app/models.py`). რაოდენობები — `data/platform.db` , 2026-10-22.

ჯგუფი	ცხრილი	დანიშნულება	მწკრივი
ვინაობა	`users`	თანამშრომელი: სახელი, `email` (unique — Google-ით შესვლისთვის), `personal_id` 🔒 (ნიღბით ჩანს), `active`	605
	`user_roles`	მომხმარებელი ↔ როლი (რამდენიმე როლი შეიძლება)	608
	`teams` , `team_members`	საველე გუნდები; წევრობა `valid_from` / `valid_to` თარიღებით	23 / 595
	`person_aliases`	სახელის ვარიანტი → მომხმარებელი (ძველი ცხრილისთვის)	591
QC ბირთვი	`qc_items`	**შემთხვევა = ოქმი + საკადასტრო კოდი**. unique: `(protocol_number, cadcode)`	44 159
	`item_people`	ოქმის ამზომველები და მიკუთვნებული მენეჯერები (`kind` , `is_primary`)	128 690
	`qc_item_codes`	ხარვეზის კოდები შემთხვევაზე (`source` : auto/manual/legacy)	45 765
	`qc_events`	**ისტორია** — მხოლოდ ემატება	44 159
	`verdict_requests`	ოპერატორის მოთხოვნა სტატუსის შესაცვლელად	0
	`qc_reviews`	QC მენეჯერის შეფასება (ძველი TYPE-ის ჩათვლით)	75
	`attachments`	ფოტო / PDF (ფაილი დისკზე, `data/uploads/`)	0
	`presence`	ვინ ხსნის ოქმს ახლა	დროებითი
განაწილება	`distribution_runs` , `distribution_presets` , `distribution_backlog`	ისტორია, შენახული წესები, ნაშთი	0 / 0 / 0
	`manager_overrides`	QC მენეჯერმა ხელით მიუწერა მენეჯერი	0
	`sporadic_imports`	სპორადულის ატვირთვები	0
ცნობარები	`defect_codes` , `missions` , `settings`	კოდები, მივლინებები, პარამეტრები	28 / 18 / 4
Pre-QC	`preqc_checks` , `preqc_findings`	ახლა სადემონსტრაციო	216 / 275
სისტემური	`sessions` , `audit_log` , `import_runs` , `emails`	სესიები, აუდიტი, იმპორტი, მეილები	—

შენიშვნა ციფრებზე:

- `qc_events` = 44 159, რადგან ძველი ცხრილიდან თითო შემთხვევას ერთი `legacy_import` ჩანაწერი აქვს. ძველი ისტორია აღდგენადი არ არის (§4.9).
- განაწილების ცხრილები ცარიელია, რადგან ბაზა ბოლოს ნულიდან აიწყო (`build_db.py`), და მას შემდეგ განაწილება არ დადასტურებულა.

კავშირები (გამარტივებული):

```
users ─┬─< user_roles
       ├─< team_members >── teams (manager_id → users)
       ├─< person_aliases
       └─< sessions, audit_log

qc_items ─┬─< item_people >── users       (ამზომველი / მენეჯერი)
          ├─< qc_item_codes >── defect_codes (ლოგიკური კავშირი კოდით)
          ├─< qc_events                    (ისტორია, actor_id → users)
          ├─< attachments                  (event_id → qc_events)
          ├─< verdict_requests
          ├── qc_reviews (1:1)
          ├── operator_id, field_manager_id, protocol_creator_id → users
          └── distribution_run_id → distribution_runs
```

`─<` ნიშნავს „ერთს ბევრი აქვს“: ერთ შემთხვევას ბევრი ისტორიის ჩანაწერი აქვს.

მარტივად რომ ავხსნათ: ბაზაში ცალკე „ყუთებია“: ადამიანები, გუნდები, ოქმები, ოქმის ისტორია, კოდები. ყუთები ერთმანეთს ნომრებით უკავშირდება. ოქმის ბარათზე წერია არა „გიორგი“, არამედ „მომხმარებელი №17“. ამიტომ სახელის გასწორება ერთ ადგილას ყველგან აისახება.

`qc_items`-ის მთავარი დროები

ველი	ძველ ცხრილში	მნიშვნელობა
`checked_at`	D	სტატუსის **პირველი გაგზავნა**. სტატისტიკის პერიოდი ამით ითვლება და შემდეგ არ იცვლება. შენახვა D-ს არ ქმნის; დაზუსტებაზე D საბოლოო სტატუსის მინიჭებისას ჩნდება
`first_sent_at`	—	პირველი გაგზავნა ნებისმიერი ვერდიქტით, დაზუსტების კითხვის ჩათვლით. ოპერატორის „დღის რაოდენობა“
`draft_saved_at`	—	შენახულია, ჯერ არ გაგზავნილა (ცისფერი ფერი)
`original_verdict`	—	პირველი გაგზავნილი სტატუსი (QC შეცდომების ანალიზისთვის)
`field_responded_at`	L	ველის პასუხის დრო
`confirmed_at` / `corrected_confirmed`	M	QC-მ გასწორება დაადასტურა
`version`	—	ერთდროული ცვლილებისგან დაცვა (§4.13)

4.5 როლები და უფლებები

7 როლი (`app/domain.py` , `app/scope.py`):

როლი	ვინ (2026-10-20)	ოქმები	Pre-QC	სტატისტიკა	მართვა
ადმინისტრატორი	ლუკა ნუკრაძე, სალომე დავითაშვილი	ყველა	✗	ყველა + შეზღუდული	მომხმარებლები, როლები, კოდები, პარამეტრები, მეილები, აუდიტი, განაწილება
QC მენეჯერი	გიორგი ჩიგოგიძე (სისტემური), ბექა სალუქვაძე (სპორადული)	ყველა	✗	ყველა + შეზღუდული	განაწილება, „არ დათანხმებულები“, სტატუსის მოთხოვნები, შეფასება
QC ოპერატორი	14	მხოლოდ მასზე დანიშნული	✗	საკუთარი	—
კოორდინატორი	დემო (ჯერ არ არის განსაზღვრული)	კონტროლის გავლილი — მხოლოდ ნახვით	✗	ყველა, შეზღუდულის გარდა	—
სამსახურის უფროსი	დემო	ბარათი არ უჩანს	✗	ხარვეზების; ოპერატორებისა და QC შეცდომების გარეშე	—
საველე მენეჯერი	14	მიკუთვნებული, **გაგზავნილი**, დადებითის გარდა	✓ თავისი გუნდი	თავისი გუნდი + ცრუ გასწორება	—
ამზომველი	247	სადაც ამზომველი ან შემდგენია — გაგზავნილი, დადებითის გარდა	✓ საკუთარი	საკუთარი	—

წესები, რომლებიც სერვერზე სრულდება:

- ველს გაუგზავნელი არაფერი უჩანს, დადებითიც არ მიდის ველზე (`scope.field_visible()`);
- ველს QC-ის შიდა ისტორია არ უჩანს: დანიშვნა, სამუშაო ვერსია, გადანაწილება, სტატუსის მოთხოვნა, შეფასება (`scope.QC_INTERNAL_EVENTS`);
- სხვისი ჩანაწერი → **404** და არა 403, რომ ჩანაწერის არსებობაც არ გამჟღავნდეს;
- ვისაც რამდენიმე როლი აქვს, ზედა ზოლში ცვლის. აქტიური როლი სესიაში ინახება, და ყოველი შემოწმება აქტიურ როლზე ხდება.

მარტივად რომ ავხსნათ: ღილაკის დამალვა საკმარისი არ არის, რადგან ცნობისმოყვარე ადამიანს მისამართის ხელით აკრეფა შეუძლია. ამიტომ სერვერი ყოველ ჯერზე თავად ამოწმებს, ვინ ითხოვს და რას. თუ ოქმი შენი არ არის, სერვერი პასუხობს „ასეთი არ არსებობს“.

4.6 სამუშაო პროცესი (state machine)

ტექნიკურად: `qc_items.state` შვიდიდან ერთ-ერთ მნიშვნელობას იღებს (`CheckConstraint ck_item_state`). გადასვლებს `workflow.apply()` მართავს, ხოლო დაშვებულ მოქმედებებს `available_actions()` ითვლის — როლის, საკუთრებისა და მდგომარეობის მიხედვით.



მდგომარეობები:

კოდი	ქართულად	ვისთანაა „ბურთი“	ახლა ბაზაში
`pending`	გასავლელი	QC ოპერატორთან	1 456
`awaiting_field`	ველზე	ამზომველებთან (მენეჯერი CC-შია)	3 774
`awaiting_manager`	„არ ვეთანხმები“ — მენეჯერის დადასტურება	პასუხისმგებელ საველე მენეჯერთან	0
`awaiting_qcm`	არ დათანხმებული	QC მენეჯერთან	290
`returned_to_manager`	დაბრუნებულია მენეჯერთან	მხოლოდ საველე მენეჯერთან	1 107
`awaiting_qc`	პასუხი მოვიდა — გადასამოწმებელი	QC ოპერატორთან	3 030
`closed`	დახურული	—	34 502

ძირითადი წესები (✅ კოდში; ბიზნეს-წესად შეთანხმებულია 2026-10-19/20):

- უარყოფითზე, რეკომენდაციასა და დაზუსტებაზე კომენტარი სავალდებულოა. კოდები კომენტარიდან ავტომატურად ისმება `rules.gs` -ის წესებით (`services/codes.classify`);
- **დაზუსტება კითხვაა და არა სტატუსი:** D არ იქმნება, დადებითად არ ითვლება, მაგრამ დღის რაოდენობაში შედის;
- გაგზავნის შემდეგ სტატუსს QC მენეჯერი პირდაპირ ცვლის, ოპერატორი კი მხოლოდ მოთხოვნით (`verdict_requests`). კომენტარი, კოდები, ამზომველები და ფოტო თავისუფლად იცვლება;
- „არ ვეთანხმები“ QC მენეჯერთან **მხოლოდ** პასუხისმგებელი (შემდგენის) მენეჯერის დადასტურებით გადადის;
- „ველი მართალია“ და „ორივე მართალია“ ვერ შეინახება, თუ „დადებითი“ არ არის მონიშნული;
- **CC პრინციპი:** დაბრუნებისას მონიშვნა „საჭიროა საველე მენეჯერის ჩართვა“. მოუნიშნავი → ამზომველებს უბრუნდება, მენეჯერი ხედავს; მონიშნული → პასუხობს მხოლოდ მენეჯერი;
- „შესწორდა“, რომელიც არ გასწორდა → `false_fix_count + 1` („ცრუ გასწორება“, ცალკე აღრიცხვა);
- თავდაპირველი ვერდიქტი არ იცვლება. უარყოფითი გასწორების შემდეგაც უარყოფითია; გამონაკლისია QC მენეჯერის „ველი მართალია“ / „ორივე მართალია“ და დადასტურებული სტატუსის შეცვლა;
- ფოტოს ტვირთავს **მხოლოდ კონტროლი** (`D.QC_ROLES`): JPEG/PNG/WEBP/GIF/PDF, ≤ 10 MB, შემთხვევითი სახელით.

მარტივად რომ ავხსნათ: ოქმი „მოგზაურობს“ ადამიანებს შორის. ყოველ მომენტში ზუსტად ცნობილია, ვისთანაა და რისი გაკეთება შეუძლია მას. ფურცელზე დახატული მარშრუტია, და სისტემა არ გაძლევს უფლებას, ამ მარშრუტიდან გადაუხვიო.

4.7 სისტემური განაწილება

ფაილები: `services/distribution/source.py` (წყარო), `algo.py` (ალგორითმი), `service.py` (გადახედვა/დადასტურება); გვერდი `/distribution` — მხოლოდ QC მენეჯერი და ადმინი.

ნაბიჯები:

1. **წყარო:** NAPR_Distributor-ის `napr_data.sqlite` , გახსნილი **მხოლოდ კითხვით** (`?mode=ro` , `source.connect()`).
2. **კანდიდატი:** კოდი შედის, თუ მისი **პირველი** ოქმი დიაპაზონშია; „სისტემური სერვისი“ იგნორირდება (`source.query_candidates()` , SQL CTE-ით). პირველ რიგში შედის წინა განაწილების **ნაშთი** (`distribution_backlog`).
3. **გამორიცხვა:**
   - ღია საველე სტატუსი;

- უკვე კონტროლშია (`qc_items.protocol_number`);
- გამორიცხული ზონები.

4. **ამზომველები და მენეჯერები** (`services/directory.FieldDirectory`):
   - OQMI_CREATOR და MOXAZ_CREATOR შედარდება Field Group-ს; ვინც იქ არ არის (მაგ. რეგისტრატორი), ამზომველად არ ითვლება;
   - მენეჯერი FIELD_GROUP_272-ით (დღევანდელი) დგინდება; ყველა მიკუთვნებული მენეჯერი ინახება, პასუხისმგებელი შემდგენის მენეჯერია;
   - თუ არცერთი ამზომველი 272-ში არ არის, ოქმი „დაუჯგუფებელშია“ და QC მენეჯერი ხელით უწერს მენეჯერს (`manager_overrides`).
5. **ბლოკი** = საკადასტრო კოდი ბოლო სეგმენტის გარეშე (`algo.block_key`); ბლოკი განუყოფელია.
6. **პროცენტული დაფარვა** (`algo.select_by_coverage`):
   - ნაგულისხმევი 70% + ზონის, სექტორის ან მენეჯერის წესი; რამდენიმე წესიდან უფრო დიდი მოქმედებს;
   - შერჩევა ფენებად ხდება (პროცენტი × ზონა × მენეჯერი), ბლოკები შემთხვევით ირჩევა (`seed` ინახება, ამიტომ შედეგი გამეორებადია).
7. **ოპერატორის რაოდენობა — მაქსიმუმი** (`algo.fit_capacity`). რიგი: ნაშთი → უფრო მაღალი პროცენტი → უფრო ძველი ოქმი. რაც არ ჩაეტია, ნაშთში რჩება.
8. **ოპერატორებზე** (`algo.assign_to_operators`): დიდი ბლოკი ჯერ, ყველაზე ნაკლებად დატვირთულს.
9. **გადახედვა → დადასტურება:**
   - გადახედვა ბაზაში არაფერს წერს; შედეგი სერვერის მეხსიერებაში 30 წუთით ინახება (`_PREVIEWS`);
   - დადასტურება ერთ ტრანზაქციაში ქმნის შემთხვევებს, ადამიანებს და „დაინიშნა“ ჩანაწერებს;
   - `uq_item_protocol_cad` ერთი ოქმის ორჯერ განაწილებას ფიზიკურად კრძალავს.

რეალური მაგალითი (დოკუმენტირებული 2026-10-20, `docs/ARCHITECTURE.md` §8.1; ამ აუდიტით თავიდან არ გაშვებულა): დიაპაზონი 24.09–18.10.2026, 14 ოპერატორი × 200, ბაზალეთი / ერთი მენეჯერის ჯგუფი 100%.

ეტაპი	კოდი
დიაპაზონში	5 840
ღია სტატუსი	-360
Field Group-ში არ არის	-238
დაუჯგუფებელი	-210
ვარგისი	5 030
შეირჩა პროცენტით	3 715
განაწილდა (14 × 200 = 2 800 მაქსიმუმი)	2 800
ნაშთი შემდეგისთვის	915

მარტივად რომ ავხსნათ: სისტემა იღებს ყველა ახლად აზომილ ნაკვეთს, აგდებს იმას, რაც ჯერ არ არის მზად ან უკვე შემოწმდა, და საჭირო ნაწილს (მაგ. 70%) ირჩევს ისე, რომ ყველა ზონა და ყველა მენეჯერი თანაბრად იყოს წარმოდგენილი. მეზობელი ნაკვეთები (ბლოკი) ერთ ოპერატორს ხვდება. რაც დღეს ვერ ჩაეტია, ხვალ პირველი შევა.

4.8 სპორადული განაწილება

`services/sporadic.py` ; გვერდი `/distribution/sporadic` .

- **წყარო:** IT-ის ყოველდღიური xlsx (ახლა მეილით მოდის), ატვირთვით. იკითხება **მხოლოდ** ეს სვეტები: `FLD_PAPER_ID, CADCODE, SURVEY_DAY, SURVEY_TIME, CALL_STATUS, ST_NAME, TR_NAME, USER_REAL_NAME, REGNUMBER, CREATE_DATE` . 🔐 განმცხადებლების სვეტები (APPLICS, APP_PERSONS, REG_EMP, ADDRESS) არ იკითხება, ატვირთული ფაილი კი დამუშავებისთანავე იშლება;
- **ამზომველი:** REGFIELDALL-ის MOXAZ_CREATOR, რადგან სპორადულზე OQMI ყოველთვის „სისტემური სერვისია“ (ძველ ჩანაწერებს 95% ემთხვევა);
- **ფილტრები:** ❓ დასაზუსტებელია (ღია კითხვა Q11 ა–დ). ამიტომ ყველა წესი გვერდზე იცვლება:
  - SURVEY_DAY: მხოლოდ გუშინ, ან გუშინ და N დღით ადრე;
  - CALL_STATUS = „დასრულებულია“;
  - ღია სტატუსის გამორიცხვა;
  - ოპერატორებზე: რაოდენობით ან თანაბრად;
- **დუბლიკატი:** სისტემურში გაუვლელი ოქმი სპორადულში გადადის (`converted_to_sporadic`); შემოწმებული აღარ ემატება.

4.9 ძველი მონაცემის გადმოტანა

`services/legacy_import.py` , `scripts/fetch_sheets.py` , `scripts/build_db.py` .

ნაბიჯი	რა ხდება
1. წამოღება	`fetch_sheets.py fetch` — კონტროლის ცხრილის ჩანართები QCStatRead Apps Script-ით, **მხოლოდ GET**, 8 000-მწკრივიანი გვერდებით → `data/raw/control/*.json`. სარკისებრი ჩანართები (`ყველა-მონაცემი` , `SQL` , …) და „პირადი ნომრები“ არ იკითხება. Field Group — Distributor-ის Apps Script-ით → `data/raw/field_group.json`
2. აწყობა	`build_db.py` — ბაზას **ნულიდან** აწყობს (≈ 2 წთ, სერვერი გაჩერებული უნდა იყოს)
3. სვეტები	სათაურით იკითხება. „სინქრონის დრო“ ორჯერ გვხვდება: პირველი = Q (კონტროლი), მეორე = S (ველი)
4. დუბლიკატი	ერთი ჩანაწერი რჩება. თუ სპორადულშიცაა — სპორადული, სხვა შემთხვევაში ყველაზე სრული
5. მდგომარეობა	E დადებითი → closed; M = TRUE → closed/corrected; ველის პასუხის შემდეგ: Q > S და O-ში შენიშვნა → `returned_to_manager` , სხვაგვარად → QC მენეჯერთან ან QC-ის გადასამოწმებელი; ცარიელი / „პროცესშია“ → pending
6. ამზომველები	F და G სვეტის სახელები, ალიასებით; ცარიელზე REGFIELDALL-იდან შეივსო 1 550 ოქმი
7. TYPE (V)	correct → „სწორია“, false defect → „არასწორი ხარვეზი“, missed defect → „გამორჩენილი“ (`qc_reviews` , `source=legacy`)
8. ისტორია	ძველი ცხრილი ზედ წერდა, ამიტომ ქრონოლოგია ვერ აღდგება. ინახება ბოლო მდგომარეობა + ნედლი ანაბეჭდი (`legacy_extra`), დროები `approx = true`

შეჯერება (დოკუმენტირებული, `ARCHITECTURE.md` §9):

- „შემაჯამებელის“ „შემოწმებული“ თითოეულ ოპერატორზე = ჩვენი რიცხვი + მოცილებული დუბლიკატები, ზუსტად (4 „პროცესშია“ ჩანაწერის გარდა);
- 271 დავა ემთხვევა QC მენეჯერთან მყოფ სისტემურ ოქმებს.

მარტივად რომ ავხსნათ: ძველი ცხრილის ყველა ჩანართი წავიკითხეთ, ცხრილს არაფერი შევუცვალეთ, და ახალ ბაზაში დავალაგეთ. ერთი ოქმი თუ ორჯერ ეწერა, ერთხელ დავტოვეთ. თითო ოქმზე დავადგინეთ, „ვისთანაა ახლა“: ველზე, მენეჯერთან, QC მენეჯერთან თუ დახურულია. მერე ჯამები ძველ შემაჯამებელს შევადარეთ, და დაემთხვა. ძველი ცხრილი მხოლოდ ბოლო მდგომარეობას ინახავდა, ამიტომ წინა ნაბიჯების ისტორია ვერ აღდგება.

⚠️ **ეს პროტოტიპის ერთჯერადი იმპორტია და არა დამტკიცებული მიგრაცია.** საწარმოო მიგრაცია ცალკე უნდა დამტკიცდეს (იხ. §7 და სახელმძღვანელოს F თავი).

4.10 სტატისტიკა

`services/stats.py` , გვერდი `/stats` , API `/api/stats` , ექსპორტი `/stats/export.xlsx` .

- **ერთი წყარო, ერთი განმარტება:** ყველა როლისთვის ერთი და იგივე ფუნქციები ითვლის. როლი მხოლოდ იმას ცვლის, რომელი ჩანაწერები შედის (`scope.item_filter`). ამიტომ ოპერატორის, მენეჯერისა და კოორდინატორის ციფრები ერთმანეთს ვერ დაშორდება;
- **მეტრიკები:**
  - შემოწმებული (D პერიოდით);
  - დადებითი (დაზუსტება არ ითვლება);
  - ხარვეზი % = უარყოფითი ÷ შემოწმებული;
  - კრიტიკული % = უარყოფითი ≥ 1 კრიტიკული კოდით ÷ შემოწმებული;
  - რეკომენდაცია (ცალკე ხაზი);
  - გასწორდა;
  - ველზე ღია;
  - პასუხის დრო;
- **ჭრილები:** ოპერატორი, მენეჯერი (ოქმი ყველა მიკუთვნებულ მენეჯერს ეთვლება), ამზომველი (ყველა ამზომველი იღებს შემთხვევას), ზონა, კოდი, სიმძიმე;
- **შეზღუდული ნაწილი** (მხოლოდ QC მენეჯერი და ადმინი): ოპერატორის შეცდომა, QC მენეჯერის შეფასებები, ძველი TYPE, „ორივე მართალია“. „ცრუ გასწორება“ — QC მენეჯერს, საველე მენეჯერს და ადმინს;
- **სიჩქარე:** SQLite-ზე 2–3 წამი (44 ათასი შემთხვევა), დოკუმენტირებული `ARCHITECTURE.md` §14-ში.

მარტივად რომ ავხსნათ: ყველა ერთი და იმავე „სასწორით“ იწონის. ოპერატორი ხედავს მხოლოდ თავის ოქმებს, მენეჯერი — თავისი გუნდისას, ხელმძღვანელობა — ყველას, მაგრამ ფორმულა ერთია. ამიტომ ერთი და იგივე ოქმი სხვადასხვა ადამიანთან სხვადასხვანაირად ვერ დაითვლება. ოპერატორების შეცდომები კი მხოლოდ მათ უჩანთ, ვისაც ეს ეხება.

4.11 Pre-QC (ახლა დემო)

`services/preqc_demo.py` , `routes/preqc.py` .

- 216 ფიქსირებული შემოწმება გამოგონილ გუნდზე, `is_demo = true` ;
- ხილვადობა **მხოლოდ** საველე მენეჯერსა და ამზომველს აქვს (`scope.can_see_preqc`). ადმინს და QC-საც კი ეკრძალება (403);
- გრაფიკი: მივლინების მე-3 დღიდან, დღე N ამოწმებს დღე N-2-ის ოქმებს (`preqc_lag_days = 2` , პარამეტრია);
- 🅘 რეალური ინტეგრაცია დაგეგმილია. ⚠️ კონტრაქტზე შეუსაბამობა იხ. §9.

4.12 მეილით შეტყობინება

`services/notify.py`.

- მოქმედებაზე მონიშვნით (✉): QC-ის დაბრუნება, QC მენეჯერის სამი გადაწყვეტილება, კომენტარი;
- ადრესატი პასუხისმგებელი მენეჯერია, CC-ში — დანარჩენი მიკუთვნებული მენეჯერები;
- SMTP-ის გარეშე მეილი `preview` სტატუსით ინახება და ჩანს: ადმინისტრირება → მეილები. სტატუსები: `preview / sent / failed / no_address`;
- ✅ ახლა ყველა მეილი `no_address` იქნება, რადგან ბაზაში ელფოსტა არავის აქვს;
- 💡 გაგზავნა მოთხოვნის შიგნით, სინქრონულად ხდება (timeout 15 წმ). საწარმოოში ფონურ რიგში უნდა გადავიდეს (§8.3).

4.13 ერთდროული მუშაობა

ტექნიკურად:

- **ოპტიმისტური ბლოკირება:** ფორმა ატარებს `version`-ს. `workflow.apply()` ადარებს მას ჩანაწერის `version`-ს და, თუ არ ემთხვევა, აბრუნებს `ConflictError`-ს. წარმატებისას `version += 1`;
- **„ახლა ხსნის“:** ბარათი ყოველ ~20 წამში აგზავნის `POST /api/presence/{id}`-ს. ჩანაწერი 60 წამის განმავლობაში ითვლება აქტიურად, 10 წუთზე ძველი კი იშლება.

მარტივად რომ ავხსნათ: თუ ორი ამზომველი ერთ ოქმს ერთდროულად ხსნის, ორივე ხედავს ყვითელ ნიშანს: „ახლა ხსნის: X“. პასუხს ის იღებს, ვინც პირველი გააგზავნა. მეორეს სისტემა ეუბნება: „უკვე უპასუხა X-მა“, და კომენტარის დამატებას სთავაზობს.

⚠️ ტექნიკური შენიშვნა იხ. §8.3 (G5): შემოწმება აპლიკაციის დონეზეა და PostgreSQL-ზე გასამკაცრებელია.

4.14 მარშრუტები (routes)

ჯგუფი	მარშრუტები	ვისთვის
შესვლა	`GET /login`, `POST /login/demo`, `POST /login/pin`, `POST /logout`, `POST /role`, `GET /auth/google`, `GET /auth/google/callback`	ყველა
მთავარი	`GET /`, `GET /about`	შესული მომხმარებელი
QC	`GET /qc`, `GET /qc/next`, `POST /qc/bulk-send`, `GET /qc/disagreements`, `GET /qc/requests`, `GET /checked`	QC როლები; „არ დათანხმებულები“ და მოთხოვნები — QC მენეჯერი/ადმინი; `/checked` — კოორდინატორი, QC მენეჯერი, ადმინი
ველი	`GET /field`	ამზომველი, საველე მენეჯერი, ადმინი
ბარათი	`GET /items/{id}`, `POST /items/{id}/action`, `GET /attachments/{id}`	ხილვადობით
განაწილება	`/distribution` (+ `preview`, `confirm`, `runs/{id}`, `presets`, `export/{token}`, `sporadic`, `sporadic/preview`, `sporadic/confirm`, `unsorted`, `unsorted/assign`, `overrides/{id}/delete`)	QC მენეჯერი, ადმინი
სტატისტიკა	`GET /stats`, `GET /api/stats`, `GET /stats/export.xlsx`	ყველა, როლის ფილტრით
Pre-QC	`GET /preqc`, `GET /preqc/{id}`	საველე მენეჯერი, ამზომველი
ადმინისტრირება	`/admin`, `/admin/users/{id}`, `/admin/teams`, `/admin/codes`, `/admin/settings`, `/admin/import`, `/admin/audit`, `/admin/emails`	ადმინი
JSON API	`GET /api/health`, `GET /api/me`, `GET /api/items`, `GET /api/items/{id}`, `POST /api/presence/{id}`, `POST /api/presence/{id}/leave`, `GET /api/presence`	ხილვადობით
დოკუმენტაცია	`GET /docs` (OpenAPI)	⚠️ ავტორიზაციის გარეშე (FastAPI-ის ნაგულისხმევი)

მარტივად რომ ავხსნათ: მარშრუტი საიტის „ოთახის მისამართია“: `/qc` ოპერატორის სიაა, `/field` ველის სია, `/admin` ადმინისტრირება. თითო ოთახს წერია, ვის შეუძლია შესვლა, და ამას სერვერი ამოწმებს. `/api/...` იგივე ოთახებია პროგრამებისთვის: გვერდის ნაცვლად მონაცემს აბრუნებს.

4.15 კონფიგურაცია

ყველაფერი გარემოს ცვლადებშია (`app/config.py`). კოდში კონკრეტული სერვერი არ წერია.

ცვლადი	ნაგულისხმევი	საწარმოოში
`QCP_DATABASE_URL`	`sqlite:///data/platform.db`	`postgresql+psycopg://…` (⚠️ დრაივერი ჯერ არ აყენია, §8.3)
`QCP_DATA_DIR`	`data/`	ფაილების საცავი

ცვლადი	ნაგულისხმევი	საწარმოოში
`QCP_SESSION_SECRET`	იქმნება `data/.session_secret`-ში	საიდუმლოების საცავიდან
`QCP_BASE_URL`	—	`https://<დომენი>`
`QCP_DEMO_LOGIN`	**`1` (ჩართული)**	**`0` სავალდებულოა**
`QCP_DEMO_PIN`	—	—
`QCP_GOOGLE_CLIENT_ID` / `_SECRET` / `_ALLOWED_DOMAIN`	—	IT-ის OAuth client
`QCP_SMTP_HOST` / `_PORT` / `_USER` / `_PASSWORD` / `_FROM` / `_STARTTLS`	— / 587 / … / `1`	IT-ის SMTP relay
`QCP_DISTRIBUTOR_DB`, `QCP_QC_WORKBOOK`	ლოკალური ფაილები	იცვლება წყაროს კონექტორით
`QCP_PORT`	8200	proxy-ს უკან

მარტივად რომ ავხსნათ: სერვერის „მისამართები და გასაღებები“ კოდში ჩაწერილი არ არის. ისინი ცალკე „ბარათზე“ (გარემოს ცვლადებში) წერია. სერვერის შეცვლისას მხოლოდ ბარათი იცვლება, კოდი კი იგივე რჩება. ⚠️ ერთი ჩანაწერი საწარმოოში აუცილებლად უნდა შეიცვალოს: `QCP_DEMO_LOGIN=0`.

4.16 ტესტები და შემოწმება

✅ `pytest -q` → **57 passed** (2026-10-22, ≈ 6 წმ). ტესტები დროებით ბაზაზე მუშაობს და `data/`-ს არ ეხება.

ფაილი	რას ამოწმებს
`test_workflow.py`	სამუშაო პროცესის ყველა გადასვლა, სავალდებულო ველები, ვერსიის კონფლიქტი
`test_scope.py`	ხილვადობა როლების მიხედვით, 404 სხვისზე
`test_distribution.py`	ალგორითმი, პროცენტები, ნაშთი, დუბლიკატის აკრძალვა
`test_legacy.py`	ძველი ცხრილის წესები (Q/S/O, დუბლიკატი, TYPE)
`test_sporadic_notify.py`	სპორადულის ფაილი და ფილტრები; მეილი პასუხისმგებელ მენეჯერს; Google-ით შესვლა მხოლოდ დარეგისტრირებული ელფოსტით (Google-თან კავშირი იმიტირებულია)
`test_auth.py`	დემო შესვლა PIN-ით და მის გარეშე; სხვა საიტიდან გაგზავნილი POST უარყოფილია (CSRF)
`test_pages.py`	გვერდები ყველა როლით იხსნება; სამსახურის უფროსს ოპერატორების ჭრილი არ უჩანს; მეორე მენეჯერი ხედავს (CC), მაგრამ ვერ ადასტურებს

დამატებით: `scripts/smoke_pages.py` (ყველა გვერდი ყველა როლით, გაშვებულ სერვერზე), `smoke_stats.py`, `smoke_distribution.py` (რეალურ წყაროზე, არაფერს წერს).

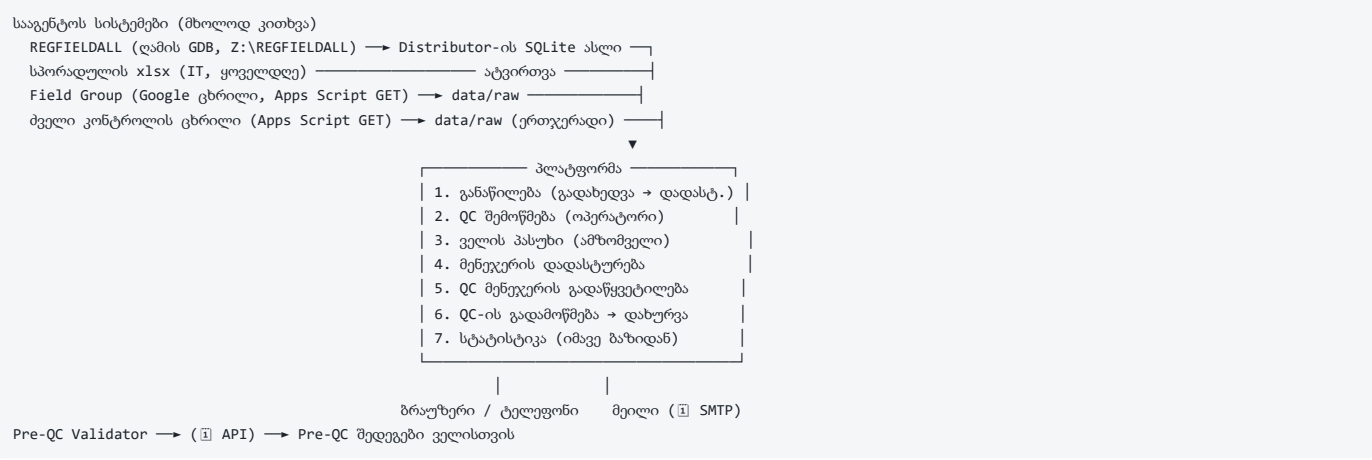
❓ **არ არის ტესტირებული:** PostgreSQL, რამდენიმე worker, დატვირთვა (load test), Google OAuth რეალურ client-ზე, SMTP.

მარტივად რომ ავხსნათ: ტესტი პატარა პროგრამაა, რომელიც სისტემას თავისით „აკლიკებს“: მაგალითად, ცდილობს, ოპერატორმა სხვისი ოქმი გახსნას, და ამოწმებს, რომ ვერ გახსნა. 57 ასეთი შემოწმება 6 წამში სრულდება და რეალურ მონაცემს არ ეხება. ჯერ არ შეგვიმოწმებია, როგორ იქცევა სისტემა ნამდვილ სერვერზე, PostgreSQL-ზე და ბევრ ერთდროულ მომხმარებელზე.

---

5. სრული ნაკადი: წყაროდან QC-ის დახურვამდე და სტატისტიკამდე

5.1 ნაკადის სქემა



მარტივად რომ ავხსნათ: მონაცემი სამი მხრიდან შემოდის: რა აიზომა (REGFIELDALL), ვინ ვისი გუნდიდანაა (Field Group) და რა ხდებოდა აქამდე (ძველი ცხრილი). საიტში ეს ყველაფერი ერთ ბაზაში ერთიანდება. შემდეგ ადამიანები ოქმს ეტაპობრივად ამუშავებენ, ყოველი ნაბიჯი ინახება და სტატისტიკა იმავე მონაცემიდან ითვლება.

5.2 რეალისტური მაგალითი — ერთი ოქმის გზა

ოქმის ნომერი და კოდი გამოგონილია; სტრუქტურა რეალურია.

#	მოქმედება	ვინ	რა იცვლება ბაზაში
1	ღამით REGFIELDALL-ის ახალი GDB ჩნდება. Distributor მას ლოკალურ SQLite-ში ტვირთავს	სისტემა / Distributor	Distributor-ის `parcels` (პლატფორმის ბაზა არ იცვლება)
2	QC მენეჯერი ირჩევს: 01.10–07.10, 14 ოპერატორი × 200, ნაგულისხმევი 70% → **გადახედვა**	QC მენეჯერი	არაფერი — მხოლოდ მეხსიერება
3	კოდი `202.43.12.42.095.042` შედის: პირველი ოქმი (№ 6150001) 03.10-ისაა, სტატუსი დახურულია, შემდგენი და მეორე ამზომველი Field Group-ში არიან. ბლოკი `202.43.12.42.095`, ზონა 43	ალგორითმი	—
4	**დადასტურება**	QC მენეჯერი	`qc_items` (state = `pending`, `operator_id`), `item_people` (2 ამზომველი + 1–2 მენეჯერი), `qc_events` (`assigned`), `distribution_runs`
5	ოპერატორი „▶ შემდეგი ოქმი“ → „უარყოფითი“ + კომენტარი „მიჯნის კოორდინატი არ ემთხვევა...“ → კოდი ავტომატურად ისმება → ფოტო → **გაგზავნა**	QC ოპერატორი	`verdict = negative`, `checked_at` (D), `first_sent_at`, `state = awaiting_field`, `qc_item_codes`, `attachments`, `qc_events` (`sent`)
6	შემდგენი ამზომველი ველის სიაში ნარინჯისფერ ოქმს ხედავს → „არ ვეთანხმები“ + დასაბუთება	ამზომველი	`field_response = disagree`, `state = awaiting_manager`, `qc_events` (`field_responded`)
7	პასუხისმგებელი მენეჯერი ადასტურებს	საველე მენეჯერი	`dispute_count = 1`, `state = awaiting_qcm`
8	QC მენეჯერი: **ხარვეზი რჩება** + ახსნა, მოუნიშნავი „საჭიროა მენეჯერის ჩართვა“, ✉ მონიშნული	QC მენეჯერი	`qcm_decision = stand`, `state = awaiting_field`, `emails` (preview / sent)
9	ამზომველი ასწორებს REDI-ში → „შესწორდა“	ამზომველი	`field_response = corrected`, `state = awaiting_qc`
10	ოპერატორი ამოწმებს → **გასწორდა — დადასტურება (M)**	QC ოპერატორი	`corrected_confirmed = true`, `state = closed`, `close_reason = corrected`. ვერდიქტი ისევ `negative`-ია
11	სტატისტიკა: ოქმი ითვლება შემოწმებულად (D = მე-5 ნაბიჯის დღე), უარყოფითად, კრიტიკულად (თუ კოდი კრიტიკულია) და „გასწორდა“-დ. ეთვლება ორივე ამზომველს და ყველა მიკუთვნებულ მენეჯერს	სისტემა	არაფერი — მხოლოდ კითხვა

ალტერნატიული ბოლო: თუ მე-8 ნაბიჯზე QC მენეჯერი „ველი მართალია“-ს ირჩევს, ოქმი იხურება `qc_error`-ით, ვერდიქტი ხდება `positive`, ხოლო ოპერატორის შეცდომა შეზღუდულ სტატისტიკაში ითვლება.

5.3 სპორადულის გზა მოკლედ

1. დილით QC მენეჯერი (სპორადული) IT-ის xlsx-ს ტვირთავს;

2. სისტემა მხოლოდ საჭირო სვეტებს კითხულობს, ფილტრავს (SURVEY_DAY, CALL_STATUS, ღია სტატუსი) და ოქმის ნომრით REGFIELDALL-ში ეძებს ამზომველს (MOXAZ_CREATOR);
3. გადახედვა → დადასტურება → `qc_items` (`kind = sporadic`). ფაილი იშლება;
4. შემდეგ ზუსტად იგივე სამუშაო პროცესია, რაც სისტემურზე.

---

6. ინტეგრაციები — რა არის დადასტურებული და რა არა

წყარო	რა გვჭირდება	დღეს როგორ მიიღება	სტატუსი	საწარმოოში (რეკომენდაცია)	სტატუსი
REGFIELDALL (საველე ოქმები)	`FLD_PAPER_ID, CADCODE, MOX_CREATE_DATE, FLD_STATUS_NAME, OQMI_CREATOR, MOXAZ_CREATOR, IS_LAST`	ღამის GDB ქსელურ საქაღალდეში `Z:\REGFIELDALL` → Distributor-ის SQLite → პლატფორმა `mode=ro`	✅	(ა) read-only view სერვისის ანგარიშით, ან (გ) ღამის ექსპორტი სერვერისთვის ხელმისაწვდომ საქაღალდეში / SFTP-ზე	💡 ❓
სპორადულის სია	ზემოთ ჩამოთვლილი 10 სვეტი, განმცხადებლების გარეშე	IT-ის xlsx მეილით → ხელით ატვირთვა	✅	(ა) view ან (გ) ყოველდღიური ფაილი SFTP-ზე	💡 ❓
Field Group (ამზომველი → მენეჯერი)	სახელი, მენეჯერი, აქტიურობა	Google ცხრილი, Apps Script Web App (GET, token-ით) → `data/raw/field_group.json`	✅	გუნდებს პლატფორმის ადმინი მართავს (`teams` , `team_members` უკვე არსებობს); ან HR/AD წყარო	💡
ძველი კონტროლის ცხრილი	ყველა ოპერატორის ჩანართი + სპორადული	QCStatRead Apps Script (GET)	✅	ერთჯერადი მიგრაცია + პარალელური პერიოდი	💡
ოქმის დეტალები (შემსრულებლები, დანართები)	„საველე სამუშაოების შესრულებაზე უფლებამოსილი პირები“	**არ მიიღება** (ამზომველი OQMI/MOXAZ-დან ითვლება)	—	API: ოქმის ნომერი → JSON	🆔 ❓
REDI (Pre-QC-ისთვის)	ოქმის გვერდი, ბარათები, ფოტოები	თანამშრომლის ბრაუზერის სესიით, გაფართოებით (`api.napr.gov.ge/lr/redi*` , `bs.napr.gov.ge`)	✅ Pre-QC-ში	სერვერისთვის API ან ექსპორტი	❓
SLR WFS (Pre-QC GIS)	ნაკვეთების გეომეტრია, მეზობლები	HTTP Basic, მომხმარებელი/პაროლი `.env` -ში (`wblr.napr.gov.ge/data/SLR/ows`)	✅ Pre-QC-ში	სერვისის ანგარიში + სერვერის IP-ის ნებართვა	❓
Google OAuth	ვინაობა (email)	კოდი მზადაა, client არ არის	🆔	IT-ის Google Cloud პროექტი, Web client	❓
SMTP	მეილის გაგზავნა	კოდი მზადაა, სერვერი არ არის	🆔	IT-ის SMTP relay, გამგზავნი მისამართი SPF/DKIM-ით	❓
Pre-QC → პლატფორმა	შემოწმების შედეგები	დემო მონაცემი	🆔	API (იხ. §9, კონტრაქტის შეუსაბამობა)	💡

მარტივად რომ ავხსნათ: დღეს მონაცემი მოდის იმიტომ, რომ ჩვენი კომპიუტერი შიდა ქსელშია, ქსელური დისკი მიერთებული აქვს და ბრაუზერში REDI-ში შესულები ვართ. სერვერს არცერთი ეს არ აქვს. ამიტომ IT-სგან თითო წყაროზე ცალკე, ოფიციალური და მხოლოდ კითხვის წვდომა გვჭირდება.

---

7. რა არის მზად, რა აკლია, რა იგეგმება

7.1 მზადაა ✅ (პროტოტიპში, რეალურ მონაცემზე)

- შვიდი როლი და ხილვადობა სერვერზე;
- სრული სამუშაო პროცესი:
  - გაგზავნა, შენახვა, დაზუსტება;
  - „არ ვეთანხმები“ მენეჯერის დადასტურებით;
  - QC მენეჯერის სამი გადაწყვეტილება;
  - CC პრინციპი;
  - სტატუსის შეცვლის მოთხოვნა;
  - QC მენეჯერის შეფასება;
- ისტორია (append-only) და აუდიტი;

- სისტემური განაწილება პროცენტებით, ნაშთით და „დაუჯგუფებლით“;
- სპორადული ფაილის ატვირთვით;
- ძველი ცხრილის გადმოტანა (44 159 შემთხვევა) შეჯერებით;
- სტატისტიკა, შეზღუდული ნაწილი, Excel ექსპორტი;
- ფოტოს ატვირთვა (მხოლოდ კონტროლი);
- „ახლა ხსნის“ ნიშანი, ვერსიის შემოწმება;
- მეილის შაბლონი და გამავალი ყუთი;
- Google OIDC-ის კოდი (Authorization Code + PKCE);
- JSON API და OpenAPI;
- ადმინისტრირება: მომხმარებლები, როლები, გუნდები, კოდები, პარამეტრები, მეილები, აუდიტი;
- LAN-ზე ჩვენება PIN-ით;
- 57 ტესტი.

7.2 დემოა ან დროებითია

- Pre-QC შედეგები (216 ფიქსირებული);
- კოორდინატორისა და სამსახურის უფროსის ანგარიშები;
- შესვლა სიიდან არჩევით;
- მეილი მხოლოდ ინახება;
- მონაცემის განახლება ხელით ხდება (`fetch_sheets.py` , `build_db.py`), და `build_db.py` ბაზას ნულიდან აწყობს.

7.3 აკლია ან დასაზუსტებელია

საკითხი	ვისზეა დამოკიდებული
სპორადულის ფილტრები (Q11 ა–დ)	ჩვენ / ხელმძღვანელობა ❓
კოორდინატორისა და სამსახურის უფროსის როლის დახვეწა და პირები	ხელმძღვანელობა ❓
თანამშრომლების ელფოსტები (ახლა 0)	მენეჯერები აგროვებენ 🅸
Pre-QC-ის შეყვანის ფორმატი (HTML / TXT / API)	ხელმძღვანელობა + IT ❓
წყაროებზე სერვერის წვდომა	IT ❓
ავტომატური ელფოსტა მონიშვნის გარეშე, დღის შეჯამება	ხელმძღვანელობა ❓

7.4 შემდეგი ფაზები (შეთავაზება)

ფაზა	შინაარსი	წინაპირობა
0. მომზადება	PostgreSQL-ზე გადასვლა, მიგრაციები, ფონური ამოცანები, უსაფრთხოების გამკაცრება (§8.3)	ჩვენი სამუშაო
1. პილოტი	სერვერი + PostgreSQL + დომენი + HTTPS + Google OAuth; მცირე ჯგუფი (QC + 1–2 გუნდი); მონაცემი ჯერ დღევანდელი გზით მოდის	IT: სერვერი, ბაზა, დომენი, OAuth
2. წყაროები	REGFIELDALL-ის view/ექსპორტი ღამის განახლებით; სპორადულის view	IT: წვდომა
3. სრული გაშვება	ყველა მომხმარებელი, SMTP, პარალელური პერიოდის შემდეგ Google ცხრილი მხოლოდ საკითხავი ხდება	შეჯერება დამთხვეულია
4. Pre-QC	Validator-ის შედეგები API-ით, ოქმის API, WFS სერვისის ანგარიშით	IT: API, WFS

---

8. შემოთავაზებული საწარმოო არქიტექტურა

8.1 დღევანდელი (პროტოტიპი) ✅

```
დეველოპერის PC (Windows)
  uvicorn (1 პროცესი, 127.0.0.1:8200 ან 0.0.0.0:8200 LAN-ზე)
    └ FastAPI აპლიკაცია
        ├ SQLite: data/platform.db (WAL)
        ├ ფაილები: data/uploads/
        ├ წყარო: ..\NAPR_Distributor\napr_data.sqlite (mode=ro)
        └ სკრიპტები ხელით: fetch_sheets.py, build_db.py
```

8.2 შემოთავაზებული (საწარმოო) 💡 ❓

```
მომხმარებელი (ოფისი / ველი, ტელეფონი)
      │ HTTPS 443 (qc.<სააგენტოს დომენი>)
      ▼
Reverse proxy (nginx / IIS / სააგენტოს load balancer) — TLS, ზომის ლიმიტი, ლოგი
      │ HTTP, localhost ან შიდა ქსელი
      ▼
აპლიკაციის სერვერი (VM, Linux ან Windows Server; სასურველია Docker)
  ├─ ვებ-პროცესი: uvicorn / gunicorn, N worker (სერვისად, ავტომატური რესტარტით)
  ├─ ფონური worker + განრიგი: ღამის იმპორტი, მეილის რიგი, presence-ის გასუფთავება
  └─ ფაილების საცავი (ფოტო/PDF) — ლოკალური დისკი ან ქსელური საცავი, ბექაპით
      │ 5432 (TLS)
      ▼
PostgreSQL 16 (იმავე ან ცალკე VM / სააგენტოს DB კლასტერი)
  ├─ აპლიკაციის მომხმარებელი (მხოლოდ ჩვენი სქემა)
  ├─ ღამის ბექაპი + WAL არქივი (PITR)
  └─ მიგრაციის მომხმარებელი (სქემის ცვლილებისთვის)

გამავალი კავშირები: Google OAuth (accounts.google.com, oauth2.googleapis.com), SMTP relay
შიდა წყაროები (მხოლოდ კითხვა): REGFIELDALL view/ექსპორტი, სპორადულის view, 🅸 ოქმის API, WFS
```

მარტივად რომ ავხსნათ:

- **Reverse proxy** სახლის კარია: დაცული შესასვლელი (HTTPS), რომელიც სტუმარს სწორ ოთახში ატარებს.
- **აპლიკაციის სერვერი** სამზარეულოა, სადაც საქმე კეთდება.
- **ფონური worker** ღამის მორიგეა: ახალ მონაცემს იღებს და მეილებს აგზავნის, ისე რომ მომხმარებელს არ ალოდინებს.
- **PostgreSQL** სეიფია, სადაც ყველა ჩანაწერი ინახება და ყოველ ღამე ასლი კეთდება.

ყველა ეს ნაწილი შეიძლება **ერთ ვირტუალურ სერვერზე** იყოს; გაყოფა სავალდებულო არ არის (იხ. სახელმძღვანელო §A.10).

8.3 რა უნდა შეიცვალოს კოდში საწარმოომდე (აუდიტის აღმოჩენები)

ესენი ✅ **დადასტურებული** ფაქტებია კოდში. გამოსავალი 💡 რეკომენდებულია, და არცერთი ჯერ არ გაკეთებულა.

#	აღმოჩენა	სად	რატომ არის მნიშვნელოვანი	შემოთავაზებული გამოსავალი
G1	PostgreSQL-ის დრაივერი (`psycopg`) `requirements.txt` -ში არ არის; კოდი PostgreSQL-ზე არასოდეს გაშვებულა	`requirements.txt`	„მხოლოდ URL-ის შეცვლა“ თეორიულად სწორია (SQLAlchemy), მაგრამ შემოწმებული არ არის	დრაივერის დამატება, ტესტების გაშვება PostgreSQL-ზე, `JSON` → `JSONB` -ის განხილვა
G2	სქემა იქმნება `Base.metadata.create_all()` -ით გაშვებისას; მიგრაციის ინსტრუმენტი არ არის	`app/main.py`	`create_all` ახალ ცხრილს ქმნის, არსებულს კი **არ ცვლის**. ველის დამატება საწარმოოში ხელით გახდებოდა საჭირო	Alembic (Pre-QC hosted-ში უკვე გამოიყენება)
G3	`QCP_DEMO_LOGIN` ნაგულისხმევად `1` -ია; დემო შესვლის cookie-ს `Secure` ნიშანი არ აქვს	`app/config.py` , `routes/auth.py`	თუ საწარმოოში ცვლადი დაგვავიწყდა, ნებისმიერს შეეძლება სიიდან ნებისმიერი ადამიანის არჩევა	ნაგულისხმევი `0` ; ან საწარმოოს რეჟიმში (`QCP_ENV=production`) დემო ფიზიკურად გაითიშოს
G4	Google-ის cookie-ზე `Secure` ჩაირთვება მხოლოდ მაშინ, როცა `request.url.scheme == "https"` ; IP აუდიტში `request.client.host` -იდან იწერება	`routes/google_auth.py` , `security.py`	proxy-ს უკან აპლიკაცია HTTP-ს ხედავს: cookie `Secure` -ის გარეშე დარჩება, აუდიტში კი proxy-ს IP ჩაიწერება	uvicorn `--proxy-headers --forwarded-allow-ips=<proxy IP>` ან `Secure` საწარმოოში ყოველთვის
G5	ვერსიის შემოწმება აპლიკაციის დონეზეა (წაკითხვა → შედარება → ჩაწერა). `UPDATE … WHERE version = ?` არ ხდება. თუ ფორმას `version` არ აქვს, შემოწმება გამოტოვდება	`workflow.apply()`	SQLite ჩაწერას თანმიმდევრულად ასრულებს, ამიტომ რისკი მცირეა. PostgreSQL-ზე რამდენიმე worker-ით ორი ერთდროული მოთხოვნა ორივე გაივლის	SQLAlchemy `version_id_col` ან `SELECT … FOR UPDATE` ; `version` სავალდებულო

#	აღმოჩენა	სად	რატომ არის მნიშვნელოვანი	შემოთავაზებული გამოსავალი
G6	განაწილებისა და სპორადულის გადახედვა სერვერის **მეხსიერებაში** ინახება (`_PREVIEWS`), დაცვა `threading.Lock` -ით	`services/distribution/service.py` , `services/sporadic.py`	რამდენიმე worker-ზე გადახედვა ერთ პროცესშია, დადასტურება კი შეიძლება მეორეში მოხვდეს → „ვადა ამოიწურა“. ლოკი მხოლოდ ერთ პროცესს იცავს	გადახედვა ბაზის ცხრილში; დადასტურებაზე advisory lock
G7	SMTP-ით გაგზავნა მოთხოვნის შიგნით, სინქრონულად ხდება (timeout 15 წმ)	`services/notify.py`	SMTP-ის შენელება მომხმარებლის ღილაკს „ჩამოკიდებს“	გამავალი რიგი (outbox) + ფონური worker, ხელახალი ცდით
G8	CSRF: მოწმდება `Origin` / `Referer` ; თუ ორივე აკლია, მოთხოვნა გადის	`security.same_origin()`	თანამედროვე ბრაუზერები ამ თავსართს აგზავნიან, მაგრამ დაცვა სრული არ არის. ეს უკვე წერია `ARCHITECTURE.md` §11-ში	CSRF token ფორმებში + `SameSite=Lax` (უკვე არის)
G9	`/docs` (OpenAPI) ავტორიზაციის გარეშე იხსნება	FastAPI-ის ნაგულისხმევი	მარშრუტების სია და სქემები ჩანს. მონაცემი არ ჩანს — API თავად ავტორიზაციას ითხოვს	საწარმოოში გამორთვა (`docs_url=None`) ან მხოლოდ ადმინისთვის
G10	დრო `datetime.now()` -ით იწერება, დროის სარტყლის გარეშე	`models._now` , `workflow` , `security`	ჩანაწერის დრო სერვერის საათის სარტყელზეა დამოკიდებული. თუ სერვერი UTC-ზეა, D-ის თარიღები 4 საათით დაიძვრება და დღის რაოდენობა აირევა	სერვერზე `Asia/Tbilisi` ან UTC-ში შენახვა + ჩვენებისას გადაყვანა; გადაწყვეტილება მიგრაციამდე
G11	მონაცემის განახლება ხელით, სკრიპტებით ხდება; `build_db.py` ბაზას ნულიდან აწყობს	`scripts/`	საწარმოოში ასეთი განახლება ყველა სამუშაოს წაშლიდა	ინკრემენტული კონექტორი (upsert) + განრიგი + იმპორტის ჟურნალი (`import_runs` უკვე არის)
G12	წყაროს მისამართები დეველოპერის გარემოზეა მიბმული: Distributor-ის ფაილი, სხვა პროექტების `config.json` (token-ებით), `Z:\`	`config.py` , `scripts/fetch_sheets.py`	სერვერზე ეს ბილიკები არ არსებობს	IT-ის ოფიციალური წყაროები (§6)
G13	ფოტოები ლოკალურ დისკზე ინახება (`data/uploads/`)	`workflow.save_files()`	რამდენიმე სერვერზე ან კონტეინერში ფაილი ერთ ადგილას უნდა იყოს; ბექაპიც სჭირდება	ერთი საცავი (ქსელური ან ობიექტური) + ბექაპი; დიდი ფაილების ზღვარი proxy-ზეც
G14	სტატისტიკის გვერდი SQLite-ზე 2–3 წამში იტვირთება	`services/stats.py`	300 მომხმარებელზე შეიძლება შენელდეს	PostgreSQL + ინდექსების შემოწმება (`EXPLAIN`), საჭიროებისას ქეში ან materialized view
G15	0 ელფოსტა ბაზაში	`users.email`	Google-ით შესვლა ვერავის შეეძლება	მენეჯერები აგროვებენ, ადმინი შეიყვანს (ან CSV იმპორტი)
G16	ლოგირება: `log_level="warning"` — წვდომის ჟურნალი პრაქტიკულად გამორთულია, სტრუქტურირებული ლოგი არ არის	`scripts/run_local.py`	ინციდენტის გამოძიებისას ვერ დავადგენთ, რა მოხდა	წვდომის ლოგი proxy-ზე (query string-ის გარეშე, რომ OAuth `code` არ ჩაიწეროს), აპლიკაციის ლოგი ფაილში ან ცენტრალურ სისტემაში
G17	პროექტის საქაღალდე git-ის რეპოზიტორია არ არის (`not a git repository` , 2026-10-22)	პროექტის ფესვი	ვერსიების ისტორია, განახლების კონტროლი და დაბრუნება არ არსებობს	git (სააგენტოს სერვერზე ან private), ვერსიის ტეგები, ცვლილებების ჟურნალი

#	აღმოჩენა	სად	რატომ არის მნიშვნელოვანი	შემოთავაზებული გამოსავალი
G18	მომხმარებლის გათიშვა (`active = false`) გუნდის წევრობას არ ხურავს, და `manager_map` წევრის აქტიურობას არ ამოწმებს	`routes/admin.user_save` , `services/seed.manager_map`	წასული ამზომველის ახალი ოქმები ძველ მენეჯერს მიეწერება, სანამ წევრობა ხელით არ დაიხურება	გათიშვისას წევრობის დახურვა (`valid_to`) ან `manager_map` -ში `tm.user.active` -ის შემოწმება; მასობრივი გადაწერის ინსტრუმენტი წასული მენეჯერისა და ოპერატორის ოქმებისთვის

მარტივად რომ ავხსნათ: პროტოტიპი ერთ კომპიუტერზე, ერთ ადამიანზე და ერთ პროცესზეა აწყობილი. ამიტომ რამდენიმე რამ მარტივად არის გაკეთებული: ბაზა ფაილშია, ზოგი რამ მეხსიერებაში ინახება, მეილი „ახლავე“ იგზავნება. სერვერზე, სადაც ბევრი ადამიანი ერთდროულად მუშაობს, ეს ნაწილები უფრო მკაცრად უნდა გაკეთდეს. სამუშაო მოცულობით პატარაა, მაგრამ IT-ს გულახდილად უნდა ვუთხრათ, რომ ის გასაკეთებელია.

---

9. აღმოჩენილი შეუსაბამობები ⚠️

#	შეუსაბამობა	წყარო A	წყარო B	როგორ ვუდგ
C1	კოორდინატორი ბარათს ხედავს თუ არა	UI: `/checked` და `scope.can_view` კოორდინატორს გავლილ ოქმებს ნახვით უჩვენებს; ტესტი `test_coordinator_reads_checked_items…` ამას ადასტურებს	API: `GET /api/items` კოორდინატორს 403-ს უბრუნებს: „კოორდინატორს მხოლოდ სტატისტიკა უჩანს“	ბიზნეს-წესი (2 10-20) — გავლ ოქმები ნახვი გასასწორებე ან როლის დახვეწისას გადაწყდება
C2	ATR-04-ის სიმძიმე	Dashboard `HANDOFF.md` (2026-08-17): ATR-04 კრიტიკული → **მნიშვნელოვანი**	პლატფორმა (2026-10-20): ATR-04 **კრიტიკულია**	ბოლო გადაწყვეტილ (10-20) მოქმედ ძველ პანელს ახალ პლატფორმაზ კრიტიკული % განსხვავებულ იქნება
C3	„მხოლოდ URL-ის შეცვლა საკმარისია PostgreSQL-ისთვის“	`config.py` , `ARCHITECTURE.md`	დრაივერი არ აყენია, ტესტი არ ყოფილა (G1)	IT-ს ვეუბნებით „SQLAlchemy ა იძლევა, გადა და ტესტირებ ჩვენი სამუშა პილოტამდე“
C4	Pre-QC-ის ინტეგრაციის კონტრაქტი	`ARCHITECTURE.md` §10: `POST /api/preqc/results` (API გასაღებით), ველები `protocol_number, cadcode, checked_on, score, status, findings[…]`	Pre-QC Hosted: `POST /api/preqc/checks` , `Authorization: Bearer pqcv_…` , idempotency `sha256(protocol_number|checked_at|validator_version|score|status)`	ერთი კონტრა უნდა შეთანხმ 💡 Hosted-ის კონტრაქტი უ სრულია (idempotency, — მისი აღება
C5	Pre-QC-ის ხილვადობა	პლატფორმა: მხოლოდ საველე მენეჯერი და ამზომველი, **ადმინიც არა**	Pre-QC Hosted: ADMIN ყველაფერს ხედავს, მიუკუთვნებელი — მხოლოდ ADMIN-ს	ბიზნეს-წესი პლატფორმის ტექნიკური ადმინის საჭი (მიუკუთვნებე ოქმები) ცალკ უნდა გადაწყ ❓

#	შეუსაბამობა	წყარო A	წყარო B	როგორ ვუდგ
C6	ამზომველის მიბმა	პლატფორმა: სახელით + ალიასებით (`name_key` , `person_aliases`)	Pre-QC Hosted: **მხოლოდ პირადი ნომრით**, სახელით მხოლოდ მაშინ, თუ ზუსტად ერთი დამთხვევაა (თანამოსახელეების რისკი)	💡 საწარმოო სახელით მიბ პირადი ნომრ უნდა გამყარ თუ წყარო მას იძლევა (Field ის API ახლა პ ნომერს არ აბრუნებს)
C7	მომხმარებლების რაოდენობა	`IT_REQUEST.md` : ≈ 300 თანამშრომელი	ბაზაში 605 მომხმარებელია (წასულების და ისტორიული სახელების ჩათვლით); აქტიური როლებით ≈ 280	IT-ს ვეუბნები 300 აქტიური, ერთდროულა (შეფასება)“
C8	ტესტების რაოდენობა	`README.md` : 57	`def test_` ფუნქცია 50-ია; parametrize-ით 57 გაშვება	შეუსაბამობა არის — 57 გაჭ ✅
C9	სტატისტიკის პანელის განთავსება	`DEPLOY.md` : Render + Neon (გარე ღრუბელი)	`OPEN_QUESTIONS.md` #5: საბოლოოდ სააგენტოს სერვერზე, „არა Render/ღრუბელი“	ახალი პლატფ სააგენტოში. პანელის მდგომარეობ გადასამოწმე ❓

10. რისკები

რისკი	ალბათობა	გავლენა	შემცირება
წყაროებზე წვდომა დიდხანს ვერ მივიღეთ	საშუალო	პილოტი ძველი გზით რჩება	პილოტი ისე იგეგმება, რომ დღევანდელი ექსპორტებით იმუშაოს; ეტაპობრივი ჩანაცვლება
ახალი და ძველი ციფრები არ დაემთხვა	საშუალო	ნდობის დაკარგვა	პარალელური პერიოდი, შეჯერების ანგარიში (მეთოდი უკვე გამოცდილია, §4.9)
სახელით მიბმის შეცდომა (თანამოსახელე, აკრეფის შეცდომა)	დაბალი–საშუალო	ოქმი სხვა ადამიანს ხვდება	ალიასების ცნობარი; „დაუჯგუფებელი“ ხელით; 💡 პირადი ნომრით მიბმა
ელფოსტები არ შეგროვდა	საშუალო	შესვლა ვერ ხერხდება	გაშვებამდე სავალდებულო ნაბიჯი
ერთი ადამიანი იცის სისტემა	მაღალი	ადმინისტრირება ჭედავს	ეს დოკუმენტები; ორი ადმინი; IT-თან ოპერაციული პასუხისმგებლობის განაწილება
პირადი მონაცემის გაჟონვა	დაბალი	სამართლებრივი	მინიმალური მონაცემი, ნიღბვა, HTTPS, როლები, აუდიტი
ერთდროული ცვლილების კონფლიქტი PostgreSQL-ზე	დაბალი	ორი პასუხი ერთ კოდზე	G5

11. ღია კითხვები (ერთ სიაში)

1. **სპორადულის ფილტრი Q11:**
   - (ა) SURVEY_DAY — მხოლოდ გუშინ, თუ გუშინ და ადრე?
   - (ბ) მხოლოდ CALL_STATUS „დასრულებული“?
   - (გ) ღია საველე სტატუსის გამორიცხვა?
   - (დ) ოპერატორებზე რაოდენობით, თუ თანაბრად? ძველი პრაქტიკა: ორშაბათს მოწმდება პარასკევი–კვირა; სტატუსით არ ფილტრავდნენ (1 690-დან 1 689 მოხვდა კონტროლში); დღიური წილი თანაბარი იყო.
2. ვინ იქნებიან კოორდინატორი და სამსახურის უფროსი, და რა უნდა დაინახონ ზუსტად?
3. Pre-QC-ის შეყვანის ფორმატი: HTML, TXT თუ API?
4. IT-ის კითხვები — დოკუმენტი 04, §3.
5. ავტომატური მეილი (მონიშვნის გარეშე) და დღის შეჯამება მენეჯერებს?
6. ძველი სტატისტიკის პანელის მომავალი (გაითიშოს? ახალი ჩაანაცვლებს?).
7. ისტორიული მონაცემის საწარმოო მიგრაცია — ვინ ამტკიცებს და რა პერიოდით?

12. ოპერაციული ინსტრუქცია (პროტოტიპი)

ამოცანა	ბრძანება	შენიშვნა
ერთჯერადი დაყენება	`setup.bat`	`.venv` + ბიბლიოთეკები + მონაცემი
გაშვება	`run.bat` → http://127.0.0.1:8200	uvicorn `--reload`-ის გარეშე
ჩვენება ტელეფონზე	`run_lan.bat`	ახალი PIN ყოველ გაშვებაზე; მხოლოდ Private ქსელი; დასრულებისას დახურეთ
მონაცემის წამოღება	`.venv\Scripts\python.exe scripts\fetch_sheets.py fetch`	მხოლოდ GET
ბაზის აწყობა	`.venv\Scripts\python.exe scripts\build_db.py`	⚠️ სერვერი გააჩერეთ; ≈ 2 წთ; **ყველა ლოკალური ცვლილება იშლება**
ტესტები	`.venv\Scripts\python.exe -m pytest -q`	დროებით ბაზაზე
შემოწმება გაშვებულ სერვერზე	`scripts\smoke_pages.py`, `smoke_stats.py`, `smoke_distribution.py`	არაფერს წერს

ცნობილი ხაფანგები:

- `.bat` ფაილები მხოლოდ ASCII უნდა იყოს. cmd.exe UTF-8-ს ვერ კითხულობს, ამიტომ ქართული შეტყობინებები Python-იდან იბეჭდება;
- სპორადულის xlsx განმცხადებლების პირად ნომრებსა და ტელეფონებს შეიცავს. მისი ასლი არსად შეინახოთ.

🔐 **მონაცემის დაცვა:** `data\` შეიცავს რეალურ სახელებს, QC კომენტარებს და პირად ნომრებს. ის არ უნდა მოხვდეს ფოსტაში, საზიარო საქაღალდეში ან git-ში (`.gitignore`-შია).

---

13. დოკუმენტების რუკა

დოკუმენტი	რისთვის
ეს (01)	რა არის, როგორ მუშაობს, რა აკლია
02 — პირადი ტექნიკური სახელმძღვანელო	ცნებების სწავლა: სერვერი, SQL, API, უსაფრთხოება, არქიტექტურა, მიგრაცია
03 — IT შეხვედრის კითხვები და პასუხები	62 კითხვა პასუხებით
04 — IT ინფრასტრუქტურის მოთხოვნები	რა გვჭირდება IT-სგან, რატომ, რა მოხდება მის გარეშე
05 — პრეზენტაციისა და საუბრის გზამკვლევი	სათქმელი ტექსტები და ჩვენების თანმიმდევრობა
06 — სწრაფი შპარგალკა	შეხვედრაზე სწრაფად მოსაძებნი
`docs/ARCHITECTURE.md`, `docs/IT_REQUEST.md`, `docs/DEMO.md`, `docs/OPEN_QUESTIONS.md`	ადრე მომზადებული დოკუმენტები (2026-10-20). ეს handoff მათ ავსებს და აუდიტის შედეგებით აზუსტებს

გენერირებულია 01-სრული-ტექნიკური-handoff.md-იდან — სწორდება მხოლოდ .md ფაილი.